SESSION FIXATION & SESSION HIJACKING

INTRODUCTION

Web applications use sessions to maintain user identity and track activity during a visit. Sessions help provide a seamless experience by eliminating the need to authenticate with every action. However, if not properly secured, session mechanisms become attractive targets for attackers. Two common session-related attacks are Session Fixation and Session Hijacking, both of which can compromise user accounts and data integrity.

1. What is a Session?

A session is a temporary and unique interaction between a user and a web server that persists across multiple requests. When a user logs into a website, the server assigns a session ID, usually stored in cookies or URLs, to identify that user until they log out or the session expires.

It helps in:

- Maintaining login state
- Storing temporary data
- Tracking user actions securely

2. What is Session Fixation?

Session Fixation is an attack where an attacker sets or “fixes” a session ID in advance and tricks a victim into authenticating with that fixed session ID.

Attack steps:

1. Attacker generates a valid session ID (before login)
2. Sends a link to the victim with this session ID
3. Victim logs in using that session
4. Attacker now has full access to the authenticated session

Mitigation: Regenerate session IDs after login to avoid reuse of pre-set session tokens.

3. What is Session Hijacking?

Session Hijacking is an attack where an attacker steals an active session ID from a user and impersonates them to gain unauthorized access.

Attack methods include:

- Packet sniffing (on insecure HTTP)
- Cross-site scripting (XSS)
- Malware or rogue browser extensions

Mitigation: Use HTTPS, set HttpOnly and Secure cookie flags, and monitor session anomalies.

4. Difference Between Session Fixation and Session Hijacking

	Session Fixation	**Session Hijacking**
When session is stolen	Before authentication	After authentication
How it's done	Attacker sets session ID for victim	Attacker steals existing session ID
User interaction	Victim logs in with attacker's session	Victim already logged in when hijack happens
Technique used	URL/session injection	Sniffing, XSS, or MITM
Mitigation	Regenerate session ID after login	Secure cookies, HTTPS, session monitoring

CONCLUSION

Understanding session-based vulnerabilities like Session Fixation and Session Hijacking is vital in securing modern web applications. Both attacks exploit weaknesses in session management and can lead to full account compromise. Through this task, I learned how these attacks work, their differences, and how to effectively mitigate them through secure coding practices and proper session handling.